Practice 10. Graph Representation

[CSE2010] Data Structures

Department of Data Science

Overview

- Construct **directed graphs** with **adjacency lists** with n vertices and m weighted edges.
- Given two vertices, check if they are adjacent.
- Given a vertex, retrieve the vertices adjacent to that vertex.

Input

- Vertices and directed and weighted edges in a **directed graph**.

- **C<space>[number_of_vertices]<space>[number_of_edges]**
  [vertex_id]<space>[vertex_id]<space>[weight]
  ⋮
  [vertex_id]<space>[vertex_id]<space>[weight]
  [number_of_edges]
  - If the construction fails, immediately terminate the program with the error message.
  - **Given n vertices, assume that vertex IDs are given as 0, 1, …, n-1.**

Input

- **I<space>[vertex_id1]<space>[vertex_id2]**
  - If there is an edge from [vertex_id1] to [vertex_id2], write “[vertex_id1]<space>[vertex_id2]<space>T” into output file.
  - Otherwise, write “[vertex_id1]<space>[vertex_id2]<space>F” into output file.

- **N<space>[vertex_id]**
  - Write into output file every vertex v adjacent to [vertex_id] such that edge ([vertex_id], v) exists in the graph.

Input and Output

- Input File

```
C 3 5
0 1 5
0 2 4
1 0 1
1 2 3
2 1 7
I 0 2
I 2 0
N 0
N 2
```

Output File

```
0 2 T
2 0 F
1 2
1
```

Running Example

- You can implement your code based on the skeleton code provided.

```
[hjkim@localhost graphs]$ python graph.py input5.txt output.txt
0: (1, 5) (2, 4)
1: (0, 1) (2, 3)
2: (1, 7)
[hjkim@localhost graphs]$ cat input5.txt
C 3 5
0 1 5
0 2 4
1 0 1
1 2 3
2 1 7
I 0 2
I 2 0
N 0
N 2
[hjkim@localhost graphs]$ cat output.txt
0 2 T
2 0 F
1 2
1
```

Submission Guideline

- Submission: **source code, an input file, an output file**
  - Deadline: **23:59, May 12th (Sunday)**

Hints

- How do you build an adjacency list?
  - a 2-D array (list) of |V| arrays (lists)
  - For each vertex v in V, maintain an array (or a list) of v's neighbors